COMPUTER ORGANIZATION AND ARCHITECTURE

QUESTION BANK ANSWER'S

MODULE 1

1. Explain the functional units of a computer with a neat block diagram.

Definition

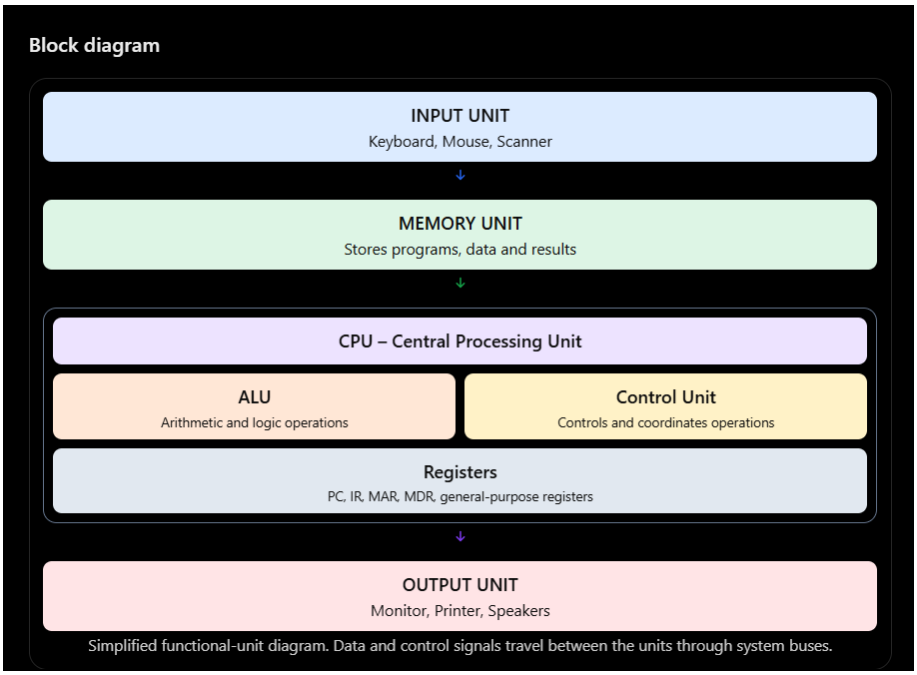
A computer is an electronic device that accepts data as input, processes it according to instructions, stores data and results, and produces output.

A computer system consists of five major functional units:

1. Input unit
2. Memory unit
3. Arithmetic and Logic Unit (ALU)
4. Control unit
5. Output unit

The ALU and Control Unit together form the CPU (Central Processing Unit).

Block diagram



Functional units

1. Input unit

The input unit accepts data and instructions from the user and converts them into a form that the computer can process.

Functions:

- Accepts data from input devices.
- Converts input into binary form.
- Transfers data to memory or CPU.

Examples: Keyboard, mouse, scanner.

2. Memory unit

The memory unit stores data, instructions, and intermediate results.

Types:

- Primary memory: RAM, ROM
- Secondary storage: SSD, HDD

Functions:

- Stores programs and data.
- Supplies instructions and data to the CPU.

- Stores intermediate and final results.

3. Arithmetic and Logic Unit (ALU)

The ALU performs arithmetic and logical operations.

Arithmetic operations:

- Addition
- Subtraction
- Multiplication
- Division

Logical operations:

- AND
- OR
- NOT
- XOR
- Comparison

4. Control unit (CU)

The control unit directs and coordinates the operations of the computer.

Functions:

- Fetches instructions from memory.
- Decodes instructions.
- Generates control signals.
- Coordinates the ALU, memory, registers, and I/O units.

5. Output unit

The output unit presents processed information to the user.

Functions:

- Receives processed results.
- Converts binary results into a suitable output form.
- Displays or produces information.



Examples: Monitor, printer, speakers.

Conclusion

All functional units work together to execute instructions. The input unit supplies data, memory stores it, the CPU processes it, and the output unit presents the result.

2. Describe the basic operational concepts of a computer with a suitable example.

Definition

The basic operational concept of a computer explains how the CPU fetches instructions from memory, decodes them, and executes them using registers, the ALU, and memory.

A program consists of a sequence of machine instructions stored in memory.

Important CPU registers

Register	Function
PC – Program Counter	Holds address of the next instruction
IR – Instruction Register	Holds the current instruction
MAR – Memory Address Register	Holds the memory address being accessed
MDR – Memory Data Register	Holds data read from or written to memory
R0, R1, etc.	General-purpose registers
ALU	Performs arithmetic and logical operations

Example: Add two numbers

Assume:

- Memory location 200 contains 5.
- Memory location 204 contains 7.
- Register R1 is used to store the result.

Instruction sequence:

LOAD R1, 200

ADD R1, 204

Step 1: Fetch instruction

The PC contains the address of the first instruction.

The CPU fetches:

LOAD R1, 200

The instruction is placed in the IR.

Step 2: Decode instruction

The control unit identifies the instruction as LOAD.

It determines that the data must be loaded from memory location 200 into R1.

Step 3: Execute LOAD

R1 ← M[200]

R1 ← 5

Step 4: Fetch and decode ADD

The CPU fetches:

ADD R1, 204

The control unit identifies the ADD operation.

Step 5: Execute ADD

R1 ← R1 + M[204]

R1 ← 5 + 7

R1 ← 12

Step 6: Store result

The result 12 is available in R1.

Conclusion

The computer executes instructions through the fetch, decode, operand-fetch, execute, and result-store stages. The PC, IR, registers, memory, control unit, and ALU cooperate during execution.

3. Explain the representation of signed numbers using sign-magnitude, 1's complement, and 2's complement methods.



Definition

Signed number representation is used to represent both positive and negative binary numbers.

For an n-bit number, the most significant bit (MSB) is commonly used to indicate the sign.

- 0 indicates positive.
- 1 indicates negative.

We will use 8-bit representation to demonstrate the methods.

Example: Represent +5 and −5.

1. Sign-magnitude representation

In sign-magnitude representation:

- MSB represents the sign.
- Remaining bits represent the magnitude.

Number	8-bit representation
+5	00000101
−5	10000101

Advantages

- Simple to understand.
- Positive and negative values have clear sign bits.

Disadvantages

- Has two representations of zero: +0 and −0.
- Arithmetic operations are more complicated.

2. 1's complement representation

In 1's complement, a negative number is obtained by complementing every bit of the positive number.

Example:

+5 = 00000101



Invert every bit:

-5 = 11111010

Number	8-bit representation
+5	00000101
−5	11111010

Advantages

- Negative number generation is simple.
- Subtraction can be performed using addition and complement.

Disadvantages

- Has positive zero and negative zero.
- Arithmetic requires end-around carry.

3. 2's complement representation

In 2's complement, a negative number is obtained by:

1. Inverting all bits of the positive number.
2. Adding 1 to the result.

Example:

+5 = 00000101

1's complement = 11111010

Add 1 = 11111011

Therefore:

-5 = 11111011

Number	8-bit representation
+5	00000101
−5	11111011

Advantages

- Only one representation of zero.

- Addition and subtraction are simpler.
- Widely used in modern computers.

Disadvantages

- Positive and negative ranges are not symmetric.
- The most negative number has no positive counterpart in the same bit width.

Comparison

Feature	Sign-magnitude	1's complement	2's complement
Negative formation	Set sign bit	Invert bits	Invert bits + 1
Zero representations	Two	Two	One
Arithmetic	More complex	End-around carry	Simple binary addition
Common modern use	Limited	Limited	Widely used

Conclusion: 2's complement is the most commonly used signed-number representation because arithmetic is simpler and zero has only one representation.

4. Describe memory locations, addresses, and memory operations in a computer system.

Definition

Computer memory is a storage unit that holds instructions, data, and results required by the processor.

Memory is divided into small storage units called memory locations. Each location has a unique address used by the CPU to access it.

1. Memory locations

A memory location is a storage unit in memory that can hold data.

- Each location stores a fixed number of bits.
- Common systems use byte-addressable memory.
- One byte contains 8 bits.
- Multiple bytes can be combined to store larger values.

2. Memory addresses



A memory address is a unique number assigned to a memory location.

Example:

Address	Data
1000	25
1001	30
1002	45
1003	60

Here, 1000, 1001, 1002, and 1003 are memory addresses.

Memory organization diagram

Address

Data

1000

25

1001

30

1002

45

1003

60

Each address identifies a particular byte in byte-addressable memory.

3. Memory operations

The two basic memory operations are:

A. Memory read operation

A memory read transfers data from memory to the CPU.

Steps:

1. CPU places the required address in MAR.



2. CPU sends a Read signal.
3. Memory identifies the specified location.
4. Data is transferred to MDR.
5. CPU uses the data.

MAR ← Address

MDR ← M[MAR]

B. Memory write operation

A memory write transfers data from the CPU to memory.

Steps:

1. CPU places the address in MAR.
2. CPU places data in MDR.
3. CPU sends a Write signal.
4. Memory stores the data at the specified address.

M[MAR] ← MDR

Memory operation diagram

CPU

MAR: Address | MDR: Data

Data transfer

MEMORY

Read / Write

Address, data, and control signals coordinate memory access.

Conclusion

Memory locations store data, addresses identify those locations, and read/write operations transfer data between memory and the processor.

5. List the steps needed to execute the machine instruction ADD LOCA, R0.

Definition

The instruction:

ADD LOCA, R0

means:

Add the contents of memory location LOCA to register R0 and store the result in R0.

Assume:

LOCA = 500

M[500] = 7

R0 = 10

Result:

R0 ← R0 + M[LOCA]

R0 ← 10 + 7

R0 ← 17

Step-by-step execution

Step 1: Fetch instruction

The PC contains the address of the instruction.

MAR ← PC

MDR ← M[MAR]

IR ← MDR

PC ← PC + 1

Step 2: Decode instruction

The control unit decodes:

ADD LOCA, R0

It identifies:

- Operation: ADD
- Source operand: memory location LOCA
- Destination register: R0



Step 3: Fetch operand

MAR ← LOCA

MDR ← M[MAR]

The memory operand is loaded into MDR.

MDR = 7

Step 4: Execute addition

R0 ← R0 + MDR

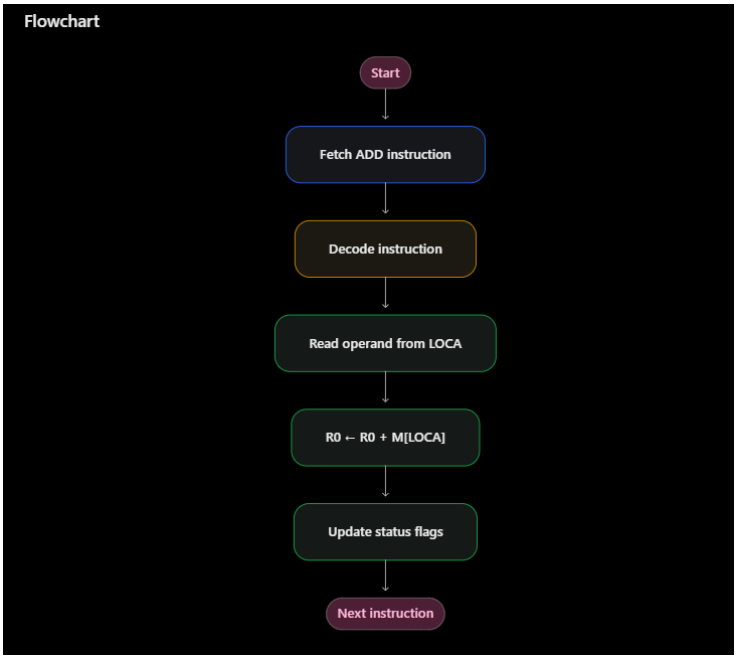
R0 ← 10 + 7

R0 ← 17

Step 5: Update status

The processor updates relevant status flags, such as zero, negative, carry, or overflow, depending on the architecture.

Flowchart



Conclusion

The ADD instruction performs instruction fetch, decoding, operand retrieval, arithmetic execution, and status update. The final result is stored in R0.

6. Convert the decimal numbers to 5-bit signed 2's complement and add. State whether overflow occurs.

Important formula

For 5-bit 2's complement:

$$ -2^{4}\leq N\leq 2^{4}-1 $$

$$ \boxed{-16\leq N\leq 15} $$

Therefore, numbers from −16 to +15 can be represented.

Conversion rule

For a negative number:

1. Convert its magnitude to 5-bit binary.
2. Invert all bits.
3. Add 1.

Overflow rule

Overflow occurs when the mathematical result is outside the range −16 to +15.

For addition:

- Positive + positive gives negative → overflow.
- Negative + negative gives positive → overflow.
- Opposite signs → no signed overflow.

(a) 5 + 10

Convert:

5 = 00101

10 = 01010

Addition:

```
  00101
+ 01010
--------
  01111
```

Result:

$$ 5+10=15 $$

No overflow

15 is the maximum representable value.

(b) −14 + 11

Convert −14:

+14 = 01110

Invert = 10001

Add 1 = 10010

11:

11 = 01011

Addition:

```
  10010
 + 01011
 --------
  11101
```

Result:

11101 = -3

\[-14+11=-3 \]

No overflow

(c) −3 + −8

Convert −3:

+3 = 00011

Invert = 11100

Add 1 = 11101

Convert −8:

+8 = 01000

Invert = 10111

Add 1 = 11000

Addition:

  11101

 + 11000

 ----

 1 10101

Discard carry beyond 5 bits:

10101 = -11

\[-3+(-8)=-11 \]

No overflow

(d) −5 + 7

Convert −5:

+5 = 00101

Invert = 11010

Add 1 = 11011

7:

7 = 00111

Addition:

  11011

 + 00111

 ----

 1 00010

Discard carry:

00010 = 2

\[-5+7=2 \]

No overflow

(e) −10 + −13

Convert −10:



+10 = 01010

Invert = 10101

Add 1 = 10110

Convert −13:

+13 = 01101

Invert = 10010

Add 1 = 10011

Addition:

```
  10110
 + 10011
 --------
 1 01001
```

5-bit result:

01001 = +9

But the actual mathematical result is:

\[-10+(-13)=-23 \]

−23 is outside the 5-bit range.

Overflow occurs

Final answer table

Case	5-bit addition	Result	Overflow
a) 5 + 10	00101 + 01010 = 01111	15	No
b) −14 + 11	10010 + 01011 = 11101	−3	No
c) −3 + −8	11101 + 11000 = 10101	−11	No
d) −5 + 7	11011 + 00111 = 00010	2	No
e) −10 + −13	10110 + 10011 = 01001	Invalid (+9 stored)	Yes

Remember: Carry-out alone does not determine signed overflow. Check the mathematical result and the sign of the operands.

7. Determine the effective address using different addressing modes.

Definition

The Effective Address (EA) is the actual memory address from which an operand is accessed or to which an operand is stored.

Different addressing modes determine how the CPU calculates the effective address.

Addressing modes

1. Immediate addressing

The operand is directly included in the instruction.

MOV R1, #25

Here:

R1 ← 25

The value 25 is the operand. No memory effective address is needed for the immediate value.

2. Direct addressing

The instruction contains the actual memory address.

Formula:

\[EA=A \]

Example:

LOAD R1, 500

If:

M[500] = 25

Then:

R1 ← M[500] = 25

EA = 500

3. Indirect addressing



The instruction contains an address that points to another memory location containing the effective address.

Formula:

\[EA=M[A] \]

Example:

A = 500

M[500] = 800

M[800] = 25

Therefore:

EA = 800

R1 ← M[800] = 25

4. Register addressing

The operand is present in a CPU register.

ADD R1, R2

Here, the operand is in R2.

No main-memory effective address is required.

5. Register indirect addressing

The register contains the memory address of the operand.

Formula:

\[EA=R \]

Example:

R2 = 500

M[500] = 25

Then:

EA = 500

6. Indexed addressing

The effective address is obtained by adding an address and an index register.

Formula:

$$EA=A+(IX)$$

Example:

A = 500

IX = 20

Therefore:

$$EA=500+20=\boxed{520}$$

7. Base register addressing

The effective address is obtained by adding the base register and displacement.

Formula:

$$EA=BR+D$$

Example:

BR = 1000

D = 40

$$EA=1000+40=\boxed{1040}$$

Summary table

Addressing mode	Effective address
Immediate	Operand is inside instruction
Direct	EA = A
Indirect	EA = M[A]
Register	Operand in register
Register indirect	EA = R
Indexed	EA = A + IX
Base register	EA = BR + D

Conclusion

Addressing modes provide different ways to access operands. They improve flexibility and allow instructions to access constants, registers, arrays, and memory locations.

8. Illustrate the instruction execution sequence for a given set of instructions.

Definition

Instruction execution is the process by which the CPU fetches, decodes, and executes instructions stored in memory.

The sequence generally consists of:

1. Fetch
2. Decode
3. Operand fetch
4. Execute
5. Store result
6. Update PC

Example instruction sequence

Assume:

M[200] = 5

M[204] = 7

Instructions:

LOAD R1, 200

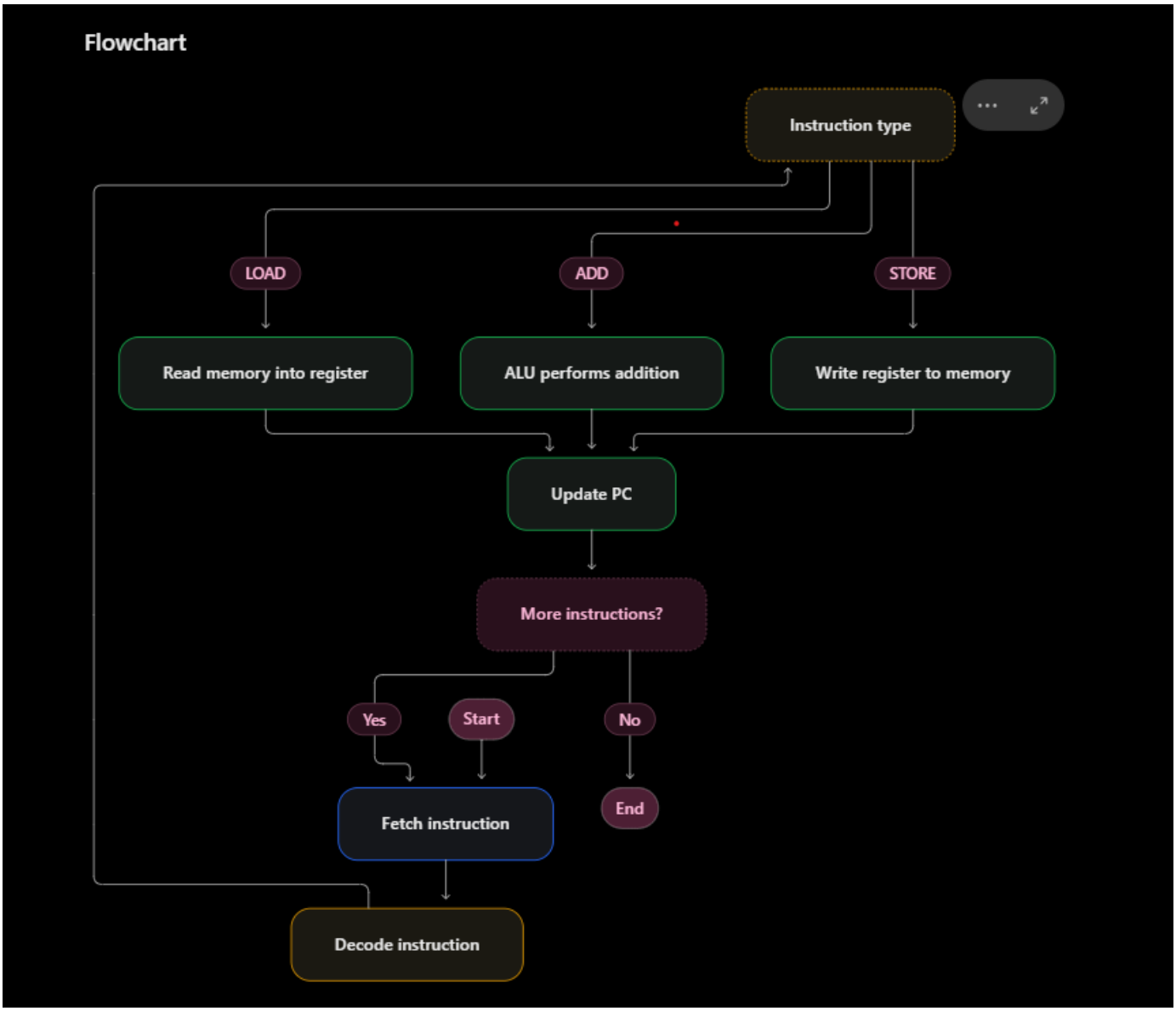
ADD R1, 204

STORE R1, 208

Execution table

Step	Instruction	Operation	Result
1	LOAD R1, 200	R1 ← M[200]	R1 = 5
2	ADD R1, 204	R1 ← R1 + M[204]	R1 = 12
3	STORE R1, 208	M[208] ← R1	M[208] = 12

Flowchart



Final result

R1 = 12

M[208] = 12

Conclusion: The CPU executes instructions one by one, using the control unit, registers, ALU, and memory.

9. Represent decimal values as signed 7-bit numbers in sign-magnitude, 1's complement, and 2's complement.

Definition

A signed 7-bit number uses:

- 1 sign bit

- 6 bits for magnitude or signed value

For sign-magnitude:

$$\text{Range}=-63\text{ to }+63$$

For 1's complement:

$$\text{Range}=-63\text{ to }+63$$

For 2's complement:

$$\text{Range}=-64\text{ to }+63$$

All given numbers are within these ranges.

Conversion rules

Sign-magnitude:

- Positive: sign bit 0.
- Negative: sign bit 1; magnitude remains unchanged.

1's complement:

- Positive: normal binary.
- Negative: invert all bits of positive representation.

2's complement:

- Positive: normal binary.
- Negative: invert all bits and add 1.

Final conversion table

Decimal	Sign-magnitude	1's complement	2's complement
+5	0000101	0000101	0000101
−2	1000010	1111101	1111110
+14	0001110	0001110	0001110
−10	1001010	1110101	1110110
+51	0110011	0110011	0110011
−43	1101011	1010100	1010101

Example: −43

Positive 43 in 7 bits:

+43 = 0101011

Sign-magnitude:

1101011

1's complement:

0101011 → 1010100

2's complement:

1010100

+1

1010101

Conclusion

Sign-magnitude stores the sign separately, 1's complement inverts the bits, and 2's complement inverts the bits and adds one. Two's complement is the most commonly used representation in modern processors.

10. Demonstrate memory read and write operations for a given sequence of instructions.

Definition

Memory read transfers data from memory to the CPU. Memory write transfers data from the CPU to memory.

Example

Assume:

M[100] = 20

M[104] = 30

Instruction sequence:

LOAD R1, 100

LOAD R2, 104



ADD R1, R2

STORE R1, 108

Step-by-step execution

Step 1: Read memory location 100

MAR ← 100

MDR ← M[100]

R1 ← MDR

Result:

R1 = 20

Step 2: Read memory location 104

MAR ← 104

MDR ← M[104]

R2 ← MDR

Result:

R2 = 30

Step 3: Perform addition

R1 ← R1 + R2

R1 ← 20 + 30

R1 ← 50

Step 4: Write result to memory

MAR ← 108

MDR ← R1

M[108] ← MDR

Result:

M[108] = 50

Memory contents

Address	Initial value	Final value
100	20	20
104	30	30
108	—	50

Conclusion

LOAD instructions perform memory reads, while STORE instructions perform memory writes. The CPU uses MAR, MDR, and control signals to transfer data.

11. Analyze the interaction among functional units during instruction execution.

Definition

The interaction among functional units describes how the input unit, memory, CPU registers, ALU, control unit, and output unit work together to execute a program.

Functional-unit interaction diagram

Input Unit

Input data and instructions

Memory Unit

Stores program and data

Control Unit

Fetches and decodes instructions

Registers

Store operands and addresses

ALU

Calculates results

Output / Memory

Displays or stores the result

Step-by-step interaction

1. Input unit: Accepts data and program instructions.
2. Memory unit: Stores the instructions and data.

3. Program Counter: Holds the address of the next instruction.
4. Control unit: Fetches and decodes the instruction.
5. Registers: Hold operands and intermediate results.
6. ALU: Performs arithmetic or logical operations.
7. Memory: Receives results when a STORE instruction is executed.
8. Output unit: Displays the final result when required.

Example

For:

ADD R1, R2

- Control unit decodes ADD.
- Registers provide R1 and R2 values.
- ALU adds them.
- Result is stored in R1.
- Status flags are updated.

Advantages of functional-unit coordination

- Faster and organized execution.
- Proper data transfer.
- Accurate instruction processing.
- Supports arithmetic, logic, and control operations.

Conclusion

The functional units cooperate through data, address, and control signals to execute machine instructions correctly.

12. Analyze the effect of different addressing modes on effective-address calculation.

Definition

An addressing mode specifies how the processor identifies the location of an operand.

The effective address is the actual memory address calculated using the instruction, registers, and displacement.



Address calculation examples

Assume:

A = 500

R1 = 100

IX = 20

BR = 1000

M[500] = 800

M[800] = 45

Addressing mode	Formula	Effective address
Immediate	Operand inside instruction	No memory EA
Direct	EA = A	500
Indirect	EA = M[A]	800
Register	Operand inside register	No memory EA
Register indirect	EA = R1	100
Indexed	EA = A + IX	520
Base register	EA = BR + displacement	Depends on displacement

Effect of addressing modes

1. Immediate addressing

The operand is part of the instruction.

ADD R1, #10

The CPU directly uses 10.

2. Direct addressing

The instruction provides the memory address.

LOAD R1, 500

EA = 500

R1 = M[500] = 800

3. Indirect addressing

The address field points to a location containing another address.

EA = M[500] = 800

R1 = M[800] = 45

4. Indexed addressing

The index register is added to the address field.

EA = 500 + 20 = 520

Useful for accessing array elements.

5. Base register addressing

The displacement is added to the base register.

EA = BR + displacement

Useful for accessing data structures and program segments.

Comparison

Mode	Main effect
Immediate	Avoids memory operand access
Direct	Simple memory access
Indirect	Supports pointer-based access
Register	Fast operand access
Register indirect	Allows register-based pointers
Indexed	Convenient for arrays
Base register	Supports relocation and structured data

Conclusion

Different addressing modes affect how operands are located and how many memory accesses may be required. Register and immediate addressing are generally fast, while indirect addressing may require additional memory access.

13. Analyze a given instruction sequence and determine the operations performed by the processor.



Definition

Instruction-sequence analysis is the process of examining instructions one by one and determining how they change registers, memory, and processor status.

Example instruction sequence

Assume:

M[200] = 10

M[204] = 5

Instructions:

LOAD R1, 200

LOAD R2, 204

SUB R1, R2

MUL R1, R2

STORE R1, 208

Step-by-step analysis

Step 1: LOAD R1, 200

R1 ← M[200]

R1 ← 10

Step 2: LOAD R2, 204

R2 ← M[204]

R2 ← 5

Step 3: SUB R1, R2

R1 ← R1 − R2

R1 ← 10 − 5

R1 ← 5

Step 4: MUL R1, R2

R1 ← R1 × R2

R1 ← 5 × 5



R1 ← 25

Step 5: STORE R1, 208

M[208] ← R1

M[208] ← 25

Final result

Register / Memory	Final value
R1	25
R2	5
M[208]	25

General method for analyzing instructions

1. Identify the initial register values.
2. Identify the initial memory values.
3. Read the first instruction.
4. Determine the source and destination operands.
5. Apply the operation.
6. Update the destination register or memory.
7. Repeat for every instruction.
8. Write the final register and memory contents.

Conclusion: Instruction analysis helps understand how a processor executes a program and changes the state of registers and memory.

14. Analyze Big Endian and Little Endian memory organization using the data COMPUTRS. Show the contents of memory words at locations 1000 and 1004.

Definition

Endianness determines the order in which bytes of a multi-byte data item are stored in memory.

There are two types:

1. Big Endian

2. Little Endian

Assume:

- Each memory word contains 4 bytes.
- The characters are stored using ASCII.
- The string is exactly COMPUTRS (8 characters).
- No null terminator is included.

ASCII values

Character	ASCII hexadecimal
C	43
O	4F
M	4D
P	50
U	55
T	54
R	52
S	53

The 8 characters occupy 2 words of 4 bytes each.

1. Big Endian

In Big Endian, the most significant byte is stored at the lowest memory address.

First word:

COM P

Bytes:

43 4F 4D 50

Second word:

UTRS

Bytes:



55 54 52 53

Big Endian memory

Word at address 1000

43

1000

4F

1001

4D

1002

50

1003

Word = 0x434F4D50

Word at address 1004

55

1004

54

1005

52

1006

53

1007

Word = 0x55545253

2. Little Endian

In Little Endian, the least significant byte is stored at the lowest memory address.

The byte order is reversed within each 4-byte word.

Little Endian memory

Word at address 1000



50

1000

4D

1001

4F

1002

43

1003

Word = 0x504D4F43

Word at address 1004

53

1004

52

1005

54

1006

55

1007

Word = 0x53525455

Final comparison

Memory word	Big Endian	Little Endian
1000	43 4F 4D 50	50 4D 4F 43
1004	55 54 52 53	53 52 54 55

Advantages and disadvantages

Big Endian	Little Endian

Byte order matches conventional most-significant-first representation	Low address contains the least significant byte
Convenient for reading hexadecimal values in memory order	Convenient for some arithmetic and low-byte access operations
Used by some network and processor architectures	Used by many modern processor architectures

Conclusion: Big Endian stores the most significant byte first, while Little Endian stores the least significant byte first. The data value is the same; only the byte arrangement differs.

15. Analyze the role of processor registers, memory, and instructions in executing a program.

Definition

A program is a collection of instructions that directs the processor to perform operations on data.

During execution:

- Registers store temporary data and addresses.
- Memory stores instructions and program data.
- Instructions specify operations to be performed.

1. Role of processor registers

Registers are small, high-speed storage locations inside the CPU.

Important registers:

Register	Role
PC	Stores the next instruction address
IR	Stores the current instruction
MAR	Stores the memory address
MDR	Stores memory data
General-purpose registers	Store operands and intermediate results

2. Role of memory



Memory stores:

- Program instructions
- Input data
- Intermediate results
- Final results

The CPU reads instructions and data from memory and writes results back when required.

3. Role of instructions

Instructions tell the processor what operation to perform.

Types:

- Data transfer: LOAD, STORE, MOV
- Arithmetic: ADD, SUB, MUL
- Logical: AND, OR, NOT
- Control: JMP, branch, CALL, RETURN

Example program

Assume:

M[100] = 8

M[104] = 2

Instructions:

LOAD R1, 100

LOAD R2, 104

ADD R1, R2

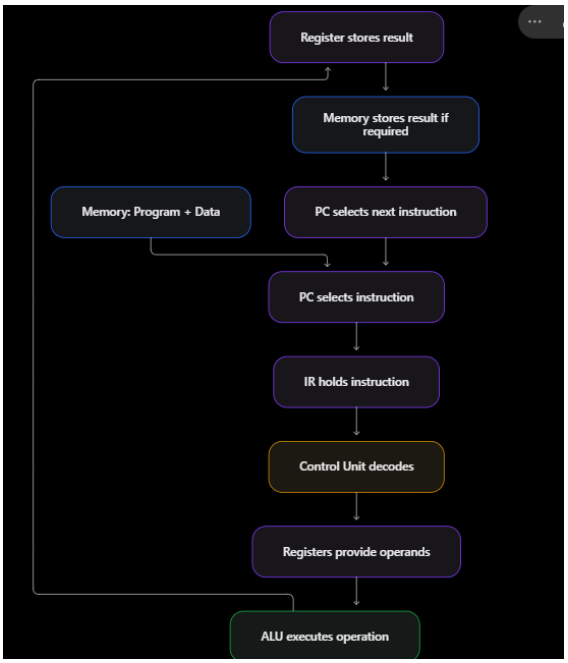
STORE R1, 108

Step-by-step execution

Step	Instruction	Register operation	Result
1	LOAD R1, 100	R1 ← M[100]	R1 = 8
2	LOAD R2, 104	R2 ← M[104]	R2 = 2

3	ADD R1, R2	R1 ← R1 + R2	R1 = 10
4	STORE R1, 108	M[108] ← R1	M[108] = 10

Execution diagram



Advantages of this organization

1. Registers provide fast access to frequently used data.
2. Memory stores large amounts of program data.
3. Instructions provide a systematic way to control operations.
4. The CPU can execute arithmetic and logical operations efficiently.
5. Register-memory interaction supports program execution.

Conclusion

Registers, memory, and instructions work together to execute programs. Memory stores instructions and data, registers hold temporary values, and instructions direct the CPU to perform operations.

MODULE 2

1. Explain the basic concepts of assembly language with suitable examples.

Definition

Assembly language is a low-level programming language that uses symbolic instructions called mnemonics to represent machine operations.

It is closely related to the machine language of a processor. Each assembly instruction generally corresponds to a machine-level operation.

Examples of mnemonics:

- MOV – move data
- ADD – addition
- SUB – subtraction
- LOAD – load data
- STORE – store data
- JMP – jump to another instruction

Basic concepts of assembly language

1. Mnemonics

Mnemonics are symbolic names used to represent machine instructions.

Example:

ADD R1, R2

This instruction adds the contents of R2 to R1.

2. Operands

Operands specify the data, registers, or memory locations used by an instruction.

Example:

MOV R1, 10

Here:

- R1 is the destination register.
- 10 is the source operand.

3. Registers

Registers are small, high-speed storage locations inside the CPU.

Examples:

- R0, R1 – general-purpose registers
- PC – Program Counter
- IR – Instruction Register
- SP – Stack Pointer

4. Labels

Labels represent symbolic names for instruction addresses.

Example:

LOOP:

ADD R1, R2

JMP LOOP

Here, LOOP identifies the address of the instruction.

5. Directives

Directives provide instructions to the assembler. They are not normally executed by the CPU.

Example:

NUM: .WORD 10

This declares a memory word named NUM containing 10 in assemblers that support this syntax.

6. Comments

Comments explain the program and are ignored by the assembler.

Example:

ADD R1, R2 ; Add R2 to R1

Example: Addition of two numbers

MOV R1, 10

MOV R2, 20

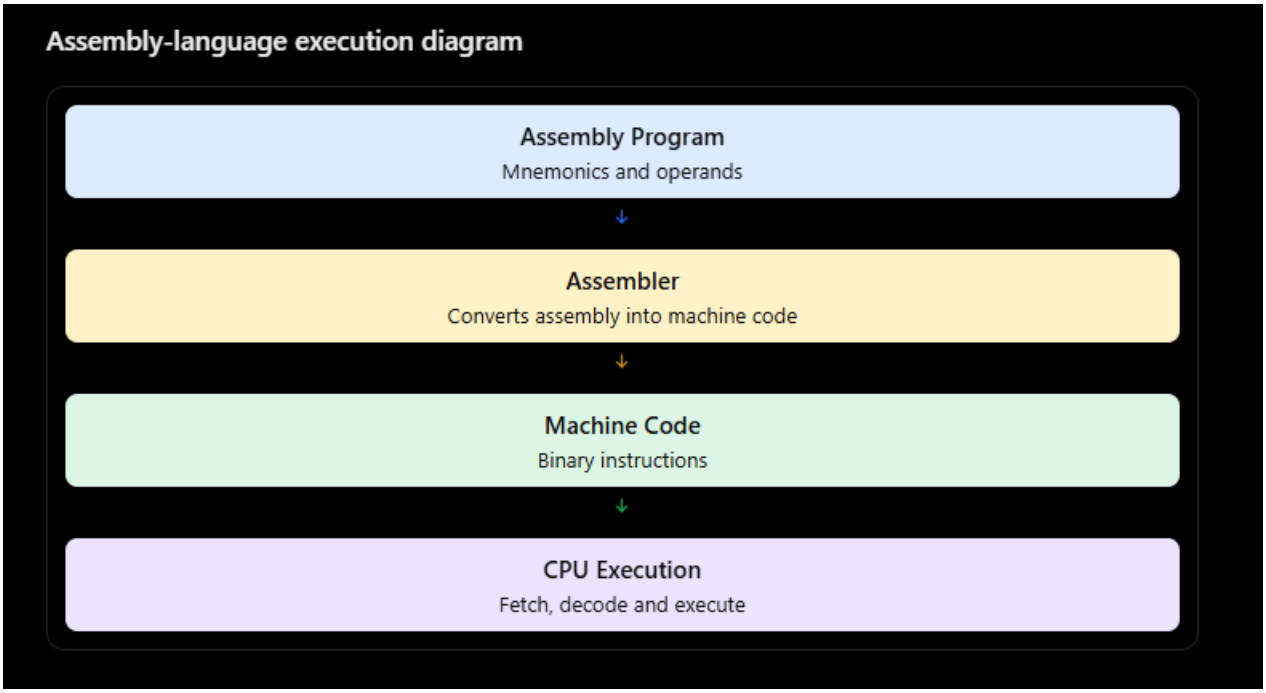
ADD R1, R2

Execution



Instruction	Operation	Result
MOV R1, 10	R1 ← 10	R1 = 10
MOV R2, 20	R2 ← 20	R2 = 20
ADD R1, R2	R1 ← R1 + R2	R1 = 30

Assembly-language execution diagram



Advantages

1. Provides direct control over processor registers.
2. Requires less memory than many high-level programs.
3. Can produce efficient machine code.
4. Useful for understanding computer architecture.
5. Suitable for low-level hardware programming.

Disadvantages

1. Difficult to learn and understand.
2. Programs are processor-dependent.
3. Requires more instructions to perform complex tasks.

4. Debugging can be difficult.
5. Less suitable for large application development.

Conclusion

Assembly language provides symbolic instructions for machine operations. It helps programmers understand CPU registers, memory, instruction execution, and low-level hardware operations.

2. Explain stack organization and the operations performed on a stack.

Definition

A stack is a memory organization in which data is inserted and removed according to the Last In, First Out (LIFO) principle.

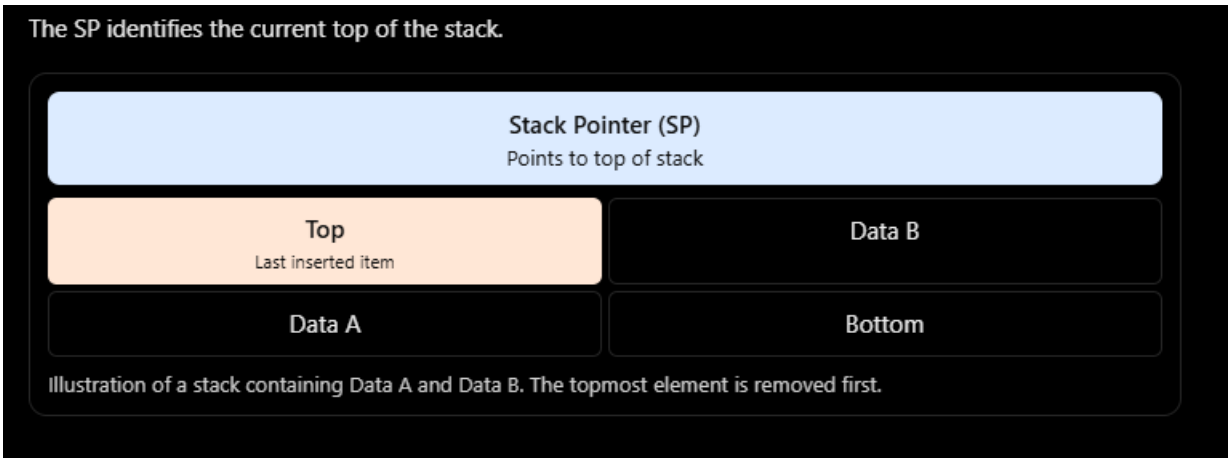
The last element inserted into the stack is the first element removed.

A stack is commonly used for:

- Subroutine calls
- Return addresses
- Temporary data
- Saving registers
- Interrupt handling

Stack organization

The stack is controlled by a register called the Stack Pointer (SP).



Stack operations

1. PUSH operation

PUSH inserts an element into the stack.

Example:

PUSH R1

This places the contents of R1 on the stack.

For a stack that grows toward lower memory addresses:

SP ← SP − 1

M[SP] ← R1

2. POP operation

POP removes the top element from the stack.

Example:

POP R2

Operation:

R2 ← M[SP]

SP ← SP + 1

The removed element is transferred into R2.

3. Stack overflow

Stack overflow occurs when an attempt is made to push data into a full stack.

4. Stack underflow

Stack underflow occurs when an attempt is made to pop data from an empty stack.

Example of PUSH and POP

Initial stack:

Empty

Instructions:

PUSH A

PUSH B

POP C



Execution:

Step	Operation	Stack contents
1	PUSH A	A
2	PUSH B	B, A
3	POP C	A

After the final instruction:

C = B

Advantages

1. Supports subroutine calls and returns.
2. Stores temporary data efficiently.
3. Supports recursive procedures.
4. Simplifies saving and restoring registers.
5. Helps manage program execution.

Disadvantages

1. Limited stack capacity.
2. Stack overflow may occur.
3. Stack underflow may occur.
4. Access is restricted by LIFO order.

Conclusion

Stack organization provides temporary storage using the LIFO principle. PUSH inserts data, POP removes data, and SP tracks the stack position.

3. Describe subroutine linkage and the sequence of operations involved in calling and returning from a subroutine.

Definition

A subroutine is a reusable sequence of instructions that performs a specific task.

Subroutine linkage is the process of transferring control from the main program to a subroutine and returning control to the main program after execution.

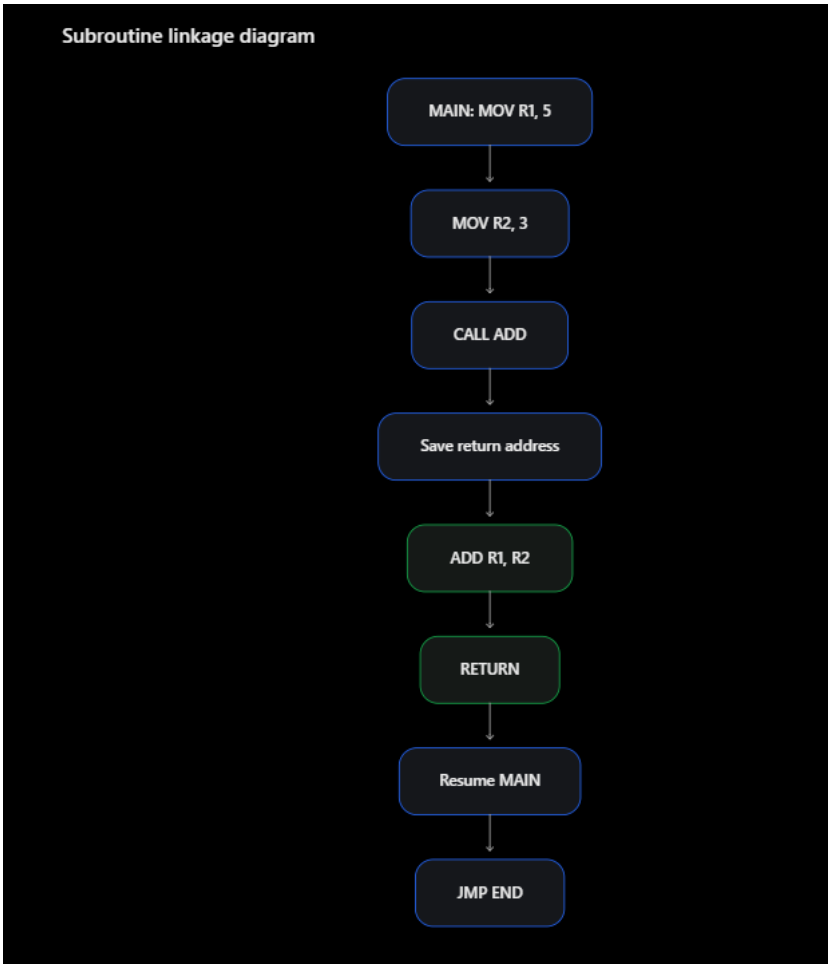


It helps avoid repeated code and improves program organization.

Important terms

- CALL: Transfers control to a subroutine.
- RETURN: Transfers control back to the calling program.
- Return address: Address of the instruction to execute after the subroutine.
- Stack: Stores return addresses and temporary data.

Subroutine linkage diagram



Sequence of operations

Step 1: Main program executes

The processor executes the main program instructions.

Step 2: CALL instruction

When the CALL instruction is encountered:

- The return address is calculated.

- The return address is saved, commonly on the stack.
- Control transfers to the subroutine address.

Step 3: Execute subroutine

The subroutine performs its required operations.

It may:

- Use registers.
- Access memory.
- Perform arithmetic.
- Save and restore registers.

Step 4: RETURN instruction

The RETURN instruction:

- Retrieves the saved return address.
- Restores control to the calling program.
- Continues execution from the instruction after CALL.

Example

```
MAIN:
  MOV R1, 10
  CALL ADDONE
  JMP END

ADDONE:
  ADD R1, 1
  RETURN

END:
```

Execution



Step	Instruction	Result
1	MOV R1, 10	R1 = 10
2	CALL ADDONE	Control transfers
3	ADD R1, 1	R1 = 11
4	RETURN	Back to main
5	JMP END	Program continues

Final result:

R1 = 11

Advantages

1. Code reusability.
2. Reduces repeated instructions.
3. Improves program readability.
4. Supports modular programming.
5. Supports recursive procedures.

Conclusion

Subroutine linkage uses CALL and RETURN instructions to transfer control between the main program and subroutine. The return address is preserved so execution can resume correctly.

4. Explain the basic methods of accessing I/O devices.

Definition

Input/Output (I/O) is the process of transferring data between the computer and external devices.

Examples:

- Keyboard
- Mouse
- Printer
- Monitor



- Disk
- Network interface

The processor communicates with I/O devices through I/O interfaces and system buses.

Basic methods of I/O

There are three major methods:

1. Programmed I/O
2. Interrupt-driven I/O
3. Direct Memory Access (DMA)

1. Programmed I/O

In programmed I/O, the processor continuously checks the status of the I/O device until it becomes ready.

This is called polling.

Steps

1. CPU sends a command to the device.
2. CPU checks the device status.
3. If the device is not ready, CPU checks again.
4. When the device is ready, data is transferred.
5. CPU continues execution.

Diagram

CPU

Checks device status repeatedly

I/O Device

Status and data registers

Advantage

- Simple to implement.
- Easy to understand.

Disadvantage



- CPU wastes time checking device status.
- Inefficient for slow devices.

2. Interrupt-driven I/O

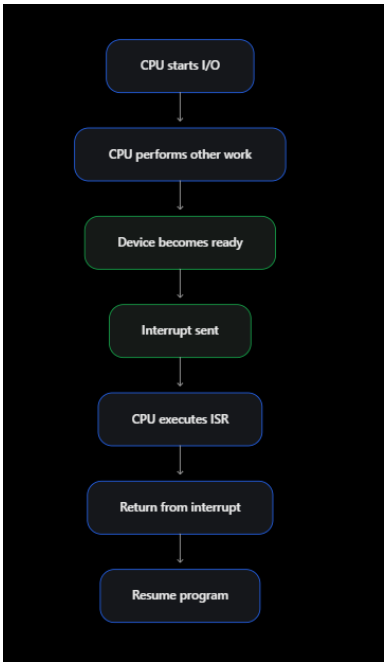
In interrupt-driven I/O, the processor starts an I/O operation and continues executing other instructions.

When the device becomes ready, it sends an interrupt to the CPU.

Steps

1. CPU initiates I/O.
2. CPU continues executing another program.
3. Device completes its operation.
4. Device sends an interrupt.
5. CPU saves its current state.
6. CPU executes the interrupt service routine (ISR).
7. CPU restores the saved state.
8. CPU resumes the interrupted program.

Diagram



Advantage

- CPU does not continuously poll the device.
- Better CPU utilization.

Disadvantage

- Interrupt handling requires additional control operations.
- Frequent interrupts can increase overhead.

3. Direct Memory Access (DMA)

DMA transfers data directly between an I/O device and main memory without requiring the CPU to move each data item individually.

A DMA controller manages the transfer.

Steps

1. CPU initializes the DMA controller.
2. CPU provides memory address, transfer size, and direction.
3. DMA controller requests the bus.
4. DMA controller transfers data between device and memory.
5. DMA controller signals completion.
6. CPU continues or handles the completion interrupt.

Diagram

CPU

Initializes DMA transfer

DMA Controller

Controls data transfer

I/O Device

Main Memory

Advantage

- Efficient for large data transfers.
- Reduces CPU involvement.



- Improves transfer performance.

Disadvantage

- Requires additional hardware.
- DMA and CPU may compete for bus access.

Comparison

Feature	Programmed I/O	Interrupt-driven I/O	DMA
CPU involvement	High	Moderate	Low for each data item
Device status	CPU polls	Device interrupts	DMA controller manages
Transfer	CPU moves data	CPU moves data in ISR	DMA transfers data
Best use	Simple transfers	Event-based I/O	Large data blocks

Conclusion

Programmed I/O uses polling, interrupt-driven I/O uses device interrupts, and DMA transfers data directly between memory and I/O devices with reduced CPU involvement.

5. Explain bus structure, bus operations, and the need for bus arbitration.

Definition

A bus is a shared communication pathway used to transfer data, addresses, and control signals between computer components.

The bus connects:

- CPU
- Memory
- I/O devices

Types of buses

1. Data bus

Transfers actual data between components.

- Usually bidirectional.
- Width determines how many bits can transfer simultaneously.



2. Address bus

Carries the address of the memory location or I/O device.

- Usually travels from the bus master to the selected target.
- Determines the addressable range.

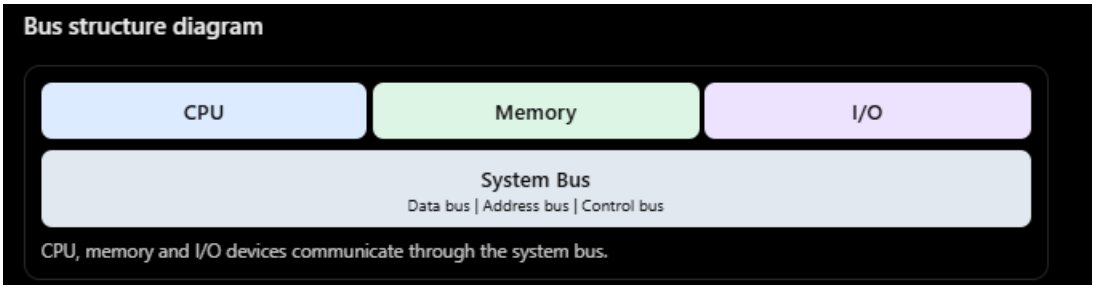
3. Control bus

Carries control signals.

Examples:

- Read
- Write
- Interrupt
- Bus request
- Bus grant

Bus structure diagram



Bus operations

1. Memory read

1. Bus master places the address on the address bus.
2. Master activates the Read control signal.
3. Memory decodes the address.
4. Memory places data on the data bus.
5. Master receives the data.

2. Memory write

1. Master places the destination address on the address bus.
2. Master places data on the data bus.

3. Master activates the Write control signal.
4. Memory stores the data at the specified address.

3. I/O read

CPU reads data from an I/O device using its interface registers.

4. I/O write

CPU sends data or commands to an I/O device through its interface.

Need for bus arbitration

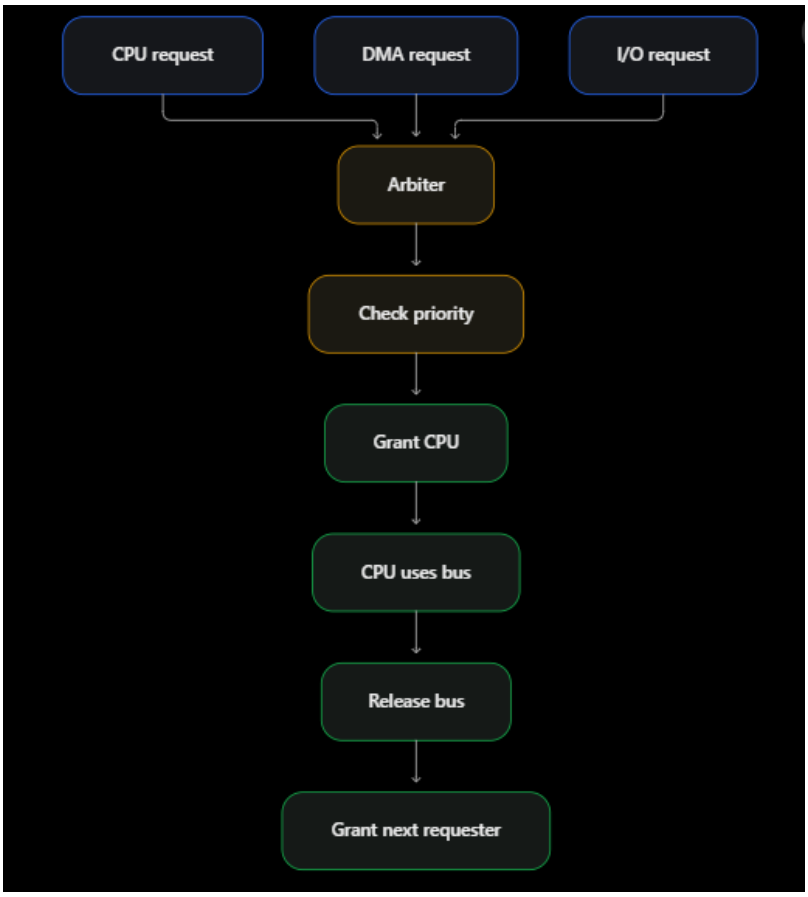
Bus arbitration is required when multiple devices request the shared bus simultaneously.

For example:

- CPU requests memory access.
- DMA controller requests memory access.
- Another bus master requests access.

Only one master can control a shared bus at a time.

Bus arbitration diagram



Advantages of bus structure

1. Reduces the number of separate connections.
2. Allows components to communicate.
3. Supports shared data transfer.
4. Simplifies system organization.

Disadvantages

1. Shared bus can become a bottleneck.
2. Multiple requests require arbitration.
3. Bus failure can affect communication.
4. Performance may decrease with many devices.

Conclusion

A system bus transfers data, addresses, and control signals. Bus arbitration ensures that only one master controls the shared bus at a time.

6. Construct an assembly-language program to perform a given arithmetic operation.

Definition

An assembly-language program uses instructions such as LOAD, ADD, SUB, and STORE to perform arithmetic operations.

The exact instructions depend on the processor's instruction set.

Example: Add two numbers and store the result

Assume:

M[100] = 15

M[104] = 25

Required operation:

\[Result=15+25 \]

Program

LOAD R1, 100

LOAD R2, 104

ADD R1, R2

STORE R1, 108

Explanation

Step 1: Load first number

R1 ← M[100]

R1 ← 15

Step 2: Load second number

R2 ← M[104]

R2 ← 25

Step 3: Add

R1 ← R1 + R2

R1 ← 15 + 25

R1 ← 40

Step 4: Store

M[108] ← R1

M[108] ← 40

Execution table

Instruction	Operation	Result
LOAD R1, 100	R1 ← M[100]	15
LOAD R2, 104	R2 ← M[104]	25
ADD R1, R2	R1 ← R1 + R2	40
STORE R1, 108	M[108] ← R1	40

Conclusion

Assembly instructions can perform arithmetic operations by loading operands into registers, executing the ALU operation, and storing the result.

7. Demonstrate stack operations for a given sequence of PUSH and POP instructions.

Definition

Stack operations follow the LIFO principle. PUSH inserts data into the stack, while POP removes the top element.

Assume a stack that grows toward lower addresses.

Initial stack:

SP = 1000

Stack is empty

Instructions:

PUSH A

PUSH B

PUSH C

POP R1

POP R2

PUSH D

POP R3

Assume:

A = 10

B = 20

C = 30

D = 40

Step-by-step execution

Step	Instruction	Stack contents (top first)	Result
1	PUSH A	A	SP = 999
2	PUSH B	B, A	SP = 998
3	PUSH C	C, B, A	SP = 997
4	POP R1	B, A	R1 = 30
5	POP R2	A	R2 = 20
6	PUSH D	D, A	SP = 998
7	POP R3	A	R3 = 40

Stack diagram after three PUSH operations

Top → C = 30

B = 20

A = 10

Last pushed element C is at the top and will be popped first.

Final register values

R1 = 30

R2 = 20

R3 = 40

Remaining stack:

A = 10

Conclusion

PUSH inserts data and POP removes the most recently inserted data. The stack pointer changes whenever an element is pushed or popped.

8. Trace the execution of an assembly-language program involving a subroutine call and return.

Definition

A subroutine is a sequence of instructions that performs a particular task.

A CALL instruction transfers control to the subroutine, and RETURN transfers control back to the main program.

Example program

```
MAIN:
   MOV R1, 5
   MOV R2, 3
   CALL ADD
   JMP END

ADD:
   ADD R1, R2
   RETURN

END:
```

Assume the CALL instruction saves the return address on the stack.

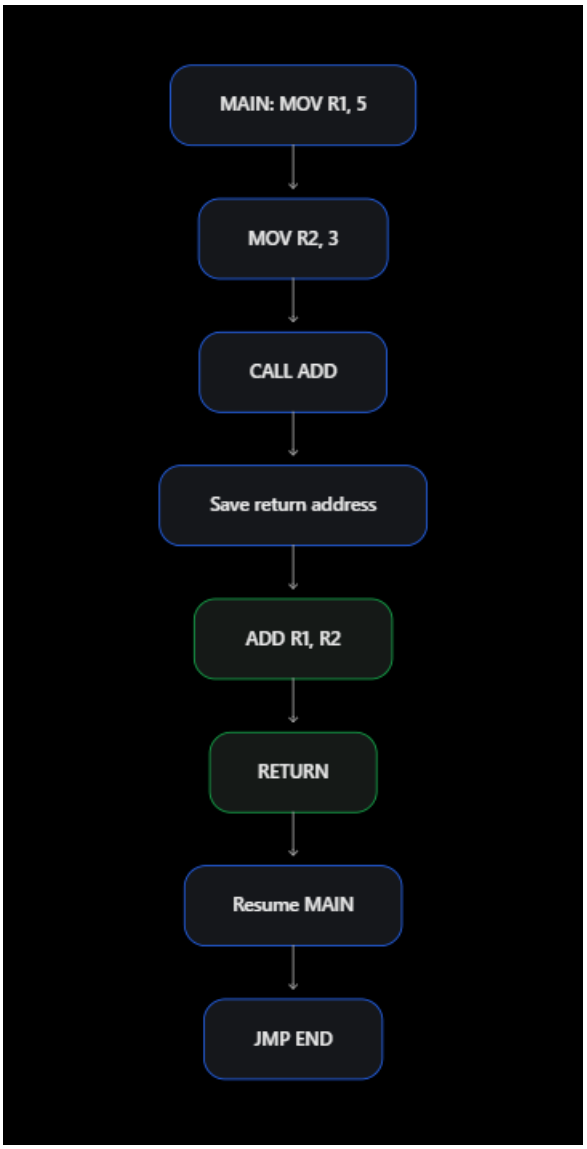
Execution trace

Step	Instruction	Operation	Result
1	MOV R1, 5	Load 5 into R1	R1 = 5
2	MOV R2, 3	Load 3 into R2	R2 = 3
3	CALL ADD	Save return address and jump	Subroutine starts



4	ADD R1, R2	R1 ← 5 + 3	R1 = 8
5	RETURN	Restore return address	Back to MAIN
6	JMP END	Transfer control to END	Program continues

Subroutine linkage diagram



Final result

R1 = 8

R2 = 3

Important points

1. CALL saves the return address.
2. Control transfers to the subroutine.

3. The subroutine executes its instructions.
4. RETURN retrieves the return address.
5. Execution continues in the main program.

9. Illustrate the sequence of operations involved in servicing an interrupt from an I/O device.

Definition

An interrupt is a signal that temporarily stops the normal execution of a program and requests the processor to handle an event.

An I/O device may generate an interrupt when:

- Data is ready.
- An operation is completed.
- An error occurs.
- The device requires CPU attention.

Interrupt servicing sequence

Step 1: I/O operation begins

The CPU initiates an operation with the I/O device.

Step 2: Device generates interrupt

When the device requires attention, it sends an interrupt request.

Step 3: CPU completes current instruction

The CPU normally completes the current instruction before servicing the interrupt.

Step 4: Save processor state

The CPU saves the return address and necessary processor state.

Step 5: Identify interrupt

The CPU determines which device generated the interrupt.

Step 6: Execute ISR

The Interrupt Service Routine handles the device event.

Step 7: Restore state

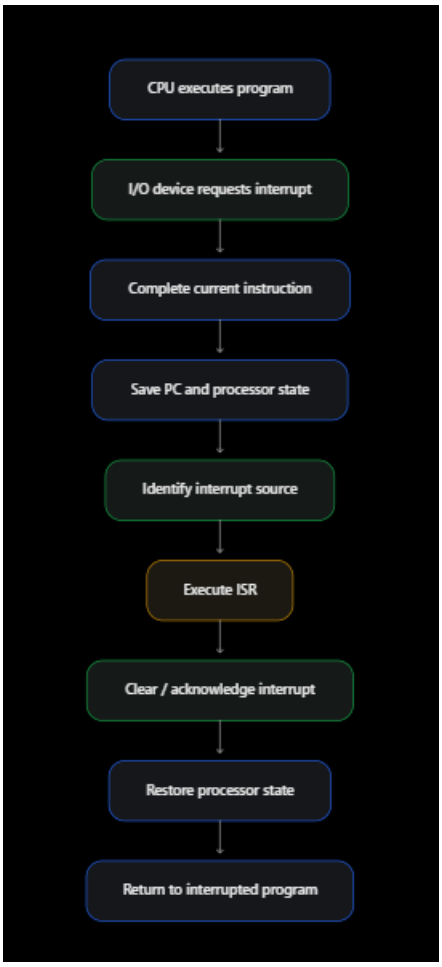
The processor restores the saved state.



Step 8: Return

The CPU resumes the interrupted program.

Interrupt flowchart



Advantages

1. CPU does not need to continuously check device status.
2. Improves CPU utilization.
3. Supports asynchronous I/O.
4. Allows quick response to device events.

Conclusion

Interrupt servicing allows an I/O device to request CPU attention. The CPU saves its state, executes the ISR, and resumes the interrupted program.

10. Apply bus arbitration principles to determine bus allocation for a given set of bus requests.

Definition

Bus arbitration is the process of selecting one bus master from multiple requesting devices.

A bus master is a device that can initiate a bus transaction.

Examples:

- CPU
- DMA controller
- I/O processor

Only one master can use a shared bus at a time.

Common arbitration methods

1. Daisy-chain arbitration

The grant signal passes through devices in sequence.

The first requesting device that receives the grant gets the bus.

Priority depends on physical position in the chain.

2. Fixed-priority arbitration

Each device has a predetermined priority.

The highest-priority requesting device gets the bus.

3. Round-robin arbitration

Devices receive bus access in a rotating order.

This provides fairer access among requesting masters.

Example

Assume fixed priority:

Priority:

CPU > DMA > I/O Controller

Requests:

CPU = 1

DMA = 1

I/O = 1



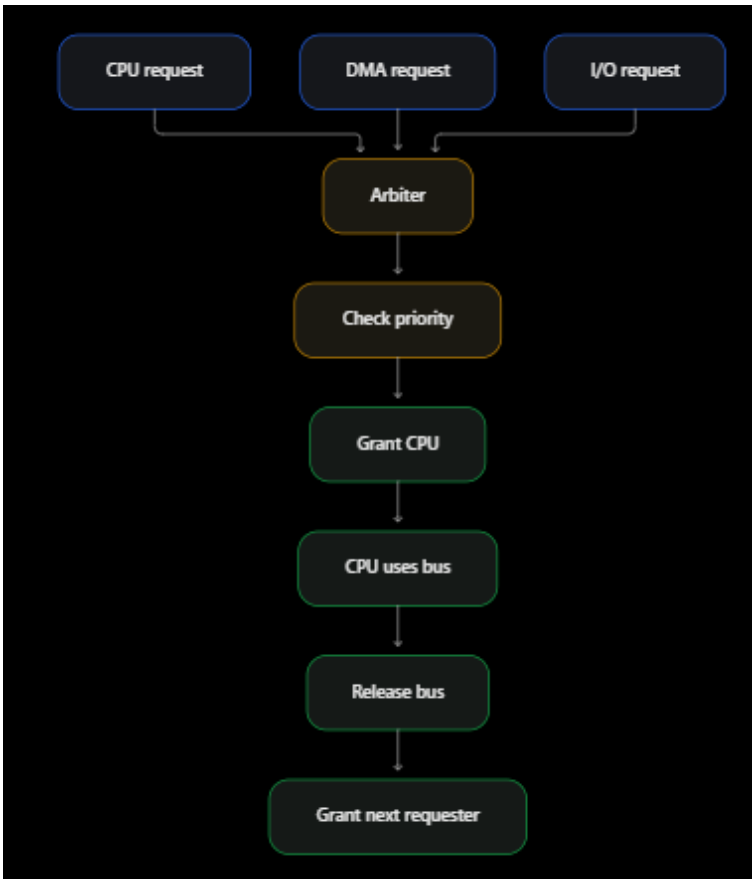
Here, 1 means requesting the bus.

Allocation

Device	Request	Priority	Result
CPU	1	Highest	Granted
DMA	1	Medium	Wait
I/O	1	Lowest	Wait

After the CPU completes its transfer and releases the bus, the arbiter selects the next requester.

Bus allocation diagram



Advantages of arbitration

1. Prevents bus conflicts.
2. Ensures controlled access.
3. Supports multiple bus masters.
4. Improves reliability of data transfer.

Conclusion

Bus arbitration determines which requesting master receives control of the bus. Fixed priority selects by priority, daisy chain selects by position, and round robin selects by rotation.

11. Analyze the execution of an assembly-language program involving stack operations and subroutine calls.

Definition

Stack operations and subroutine calls are used together to preserve return addresses and temporary data during program execution.

The stack follows the LIFO principle.

Example program

```
MAIN:
   MOV R1, 10
   PUSH R1
   CALL DOUBLE
   POP R2
   JMP END

DOUBLE:
   ADD R1, R1
   RETURN

END:
```

Assume the CALL instruction saves its return address on the stack and RETURN restores it.

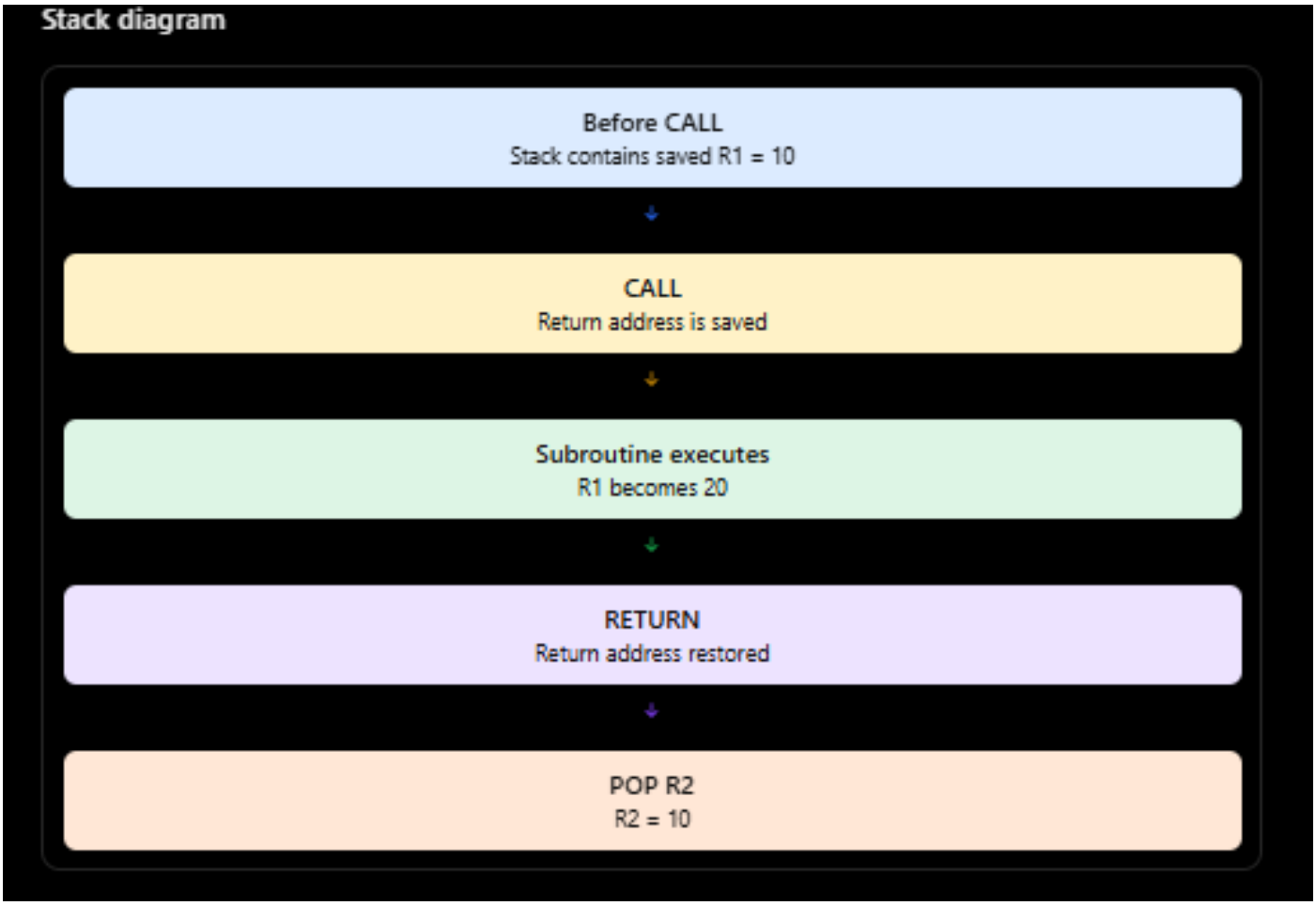
Step-by-step execution

Step	Instruction	Operation	Result
1	MOV R1, 10	Load 10 into R1	R1 = 10



2	PUSH R1	Push R1 value	Stack = 10
3	CALL DOUBLE	Save return address and jump	Subroutine begins
4	ADD R1, R1	R1 ← 10 + 10	R1 = 20
5	RETURN	Restore return address	Back to MAIN
6	POP R2	Remove top stack value	R2 = 10
7	JMP END	Continue program	End

Stack diagram



Final values

R1 = 20

R2 = 10

12. Analyze the sequence of events involved in interrupt handling during an I/O operation.

Definition

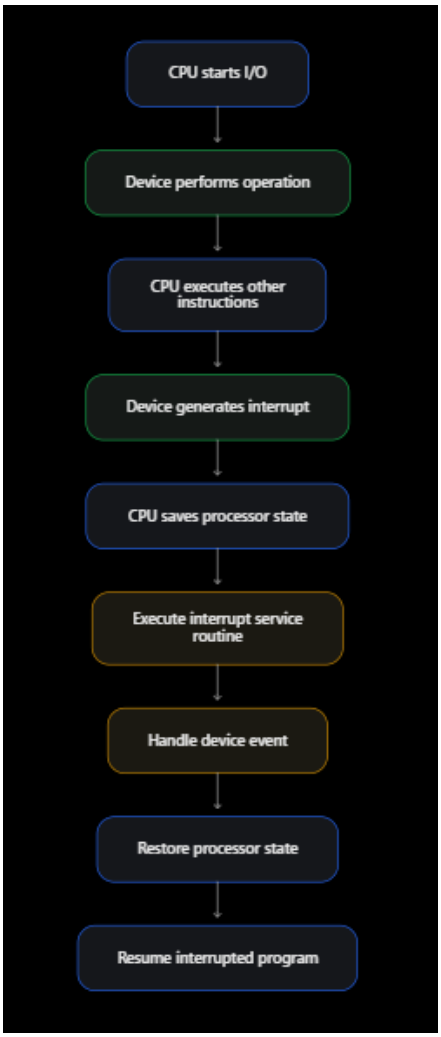
Interrupt handling is the process by which the CPU responds to an event generated by an I/O device.

It allows the device to request service without requiring the CPU to continuously monitor it.

Sequence of events

1. CPU initiates an I/O operation.
2. I/O device performs its operation.
3. CPU continues executing other instructions.
4. Device becomes ready or completes the operation.
5. Device generates an interrupt request.
6. CPU completes the current instruction.
7. CPU saves the PC and necessary processor state.
8. CPU identifies the interrupt source.
9. CPU executes the ISR.
10. ISR handles the I/O event and acknowledges the interrupt.
11. CPU restores its saved state.
12. CPU resumes the interrupted program.

Diagram



Example

A keyboard generates an interrupt when a key is pressed.

- CPU is executing a program.
- Keyboard sends an interrupt.
- CPU saves its state.
- Keyboard ISR reads the key data.
- CPU restores its state.
- Program execution resumes.

Advantages

- Efficient CPU utilization.
- Supports asynchronous events.
- Reduces continuous polling.

- Allows quick response to I/O events.

Conclusion

Interrupt handling provides a structured way for the CPU to service I/O events, execute the ISR, and resume the interrupted program.

13. Analyze the operation of programmed I/O and interrupt-driven I/O for a given data-transfer scenario.

Definition

Programmed I/O and interrupt-driven I/O are two methods used by the processor to transfer data between the CPU and I/O devices.

1. Programmed I/O

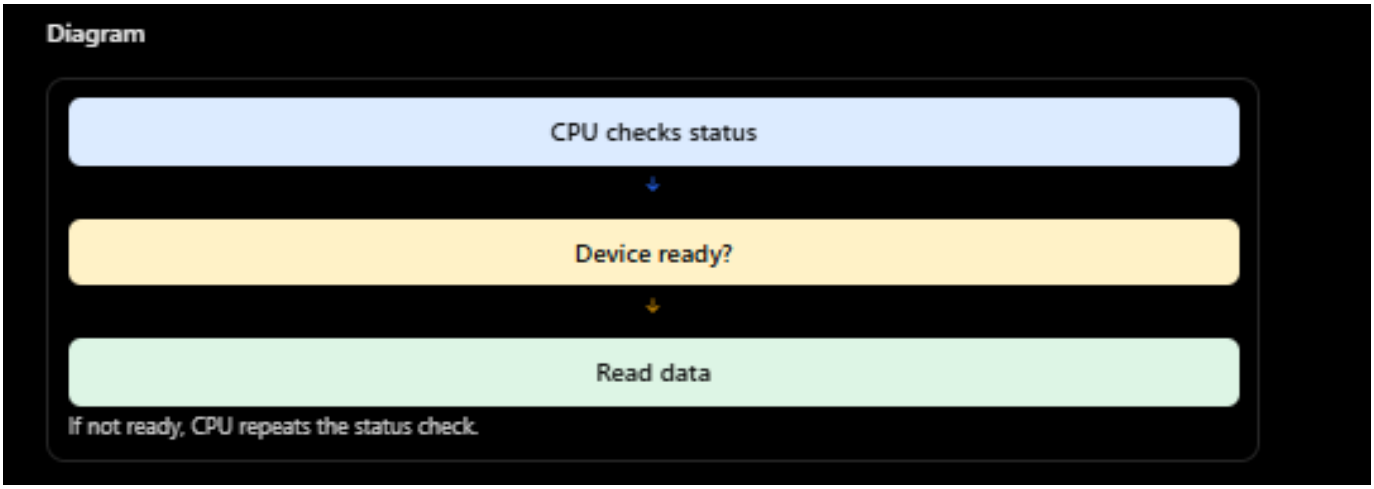
In programmed I/O, the CPU repeatedly checks the device status until it is ready.

Example

A CPU receives one character from a keyboard.

Sequence:

1. CPU checks keyboard status.
2. If the keyboard is not ready, CPU checks again.
3. Keyboard becomes ready.
4. CPU reads the character.
5. CPU continues execution.



2. Interrupt-driven I/O

In interrupt-driven I/O, the CPU starts the operation and continues executing other instructions.

When the device becomes ready, it generates an interrupt.

Example

1. CPU starts keyboard input.
2. CPU performs another task.
3. Keyboard receives a key.
4. Keyboard generates interrupt.
5. CPU executes keyboard ISR.
6. CPU reads the key.
7. CPU resumes its previous program.

Comparison

Feature	Programmed I/O	Interrupt-driven I/O
Device checking	Continuous polling	Device sends interrupt
CPU activity	Waits for device	Performs other work
Data transfer	CPU handles data	CPU handles data in ISR
CPU efficiency	Lower for slow devices	Generally better
Complexity	Simple	More complex
Suitable for	Simple or small transfers	Event-driven devices

Conclusion

Programmed I/O uses continuous status checking, while interrupt-driven I/O allows the device to notify the CPU when service is required. Interrupt-driven I/O generally improves CPU utilization.

14. Analyze the transfer of data among the processor, memory, and I/O devices using a given bus structure.

Definition

Data transfer among the processor, memory, and I/O devices takes place through a system bus.

The system bus contains:

1. Data bus
2. Address bus
3. Control bus

Example: CPU reads data from an I/O device and stores it in memory.

Assume:

I/O device data = 50

Destination memory address = 1000

Step-by-step operation

Step 1: CPU initiates I/O read

The CPU sends the I/O device address using the address bus.

Step 2: Control signal

CPU activates the I/O Read signal.

Step 3: Device provides data

The I/O interface places data 50 on the data bus.

Step 4: CPU receives data

The processor receives the data in a register.

R1 ← 50

Step 5: CPU writes data to memory

CPU places destination address 1000 on the address bus.

It places data 50 on the data bus.

The CPU activates the Memory Write signal.

Step 6: Memory stores data

M[1000] ← 50

Data-transfer diagram

Processor

Memory

I/O Device

System Bus

Address | Data | Control

I/O read: Device → Processor

Memory write: Processor → Memory

Important points

- Address bus identifies the destination.
- Data bus transfers the actual data.
- Control bus specifies the operation.
- CPU controls ordinary programmed I/O transfers.
- DMA can transfer data between memory and I/O with reduced CPU involvement.

Conclusion

The system bus provides communication between the processor, memory, and I/O devices. Address, data, and control signals coordinate each transfer.

15. Analyze a given bus arbitration scenario involving multiple bus masters.

Definition

A bus master is a device that can initiate a data transfer over the system bus.

Bus arbitration selects one master when multiple masters request the bus simultaneously.

Examples:

- CPU
- DMA controller
- I/O processor

Common bus arbitration methods

1. Fixed-priority arbitration

Each master has a fixed priority.

The highest-priority requesting master receives the bus.

2. Daisy-chain arbitration



The grant signal passes through masters in physical order.

The first requesting master that receives the grant gets bus access.

3. Round-robin arbitration

Priority rotates after each bus allocation.

This helps distribute bus access fairly.

Worked example: Fixed priority

Assume:

Master	Priority
CPU	1 – Highest
DMA	2
I/O Controller	3 – Lowest

All three request the bus simultaneously.

Step 1: Check requests

CPU = Request

DMA = Request

I/O = Request

Step 2: Apply priority

The CPU has the highest priority.

Therefore:

Bus granted to CPU

Step 3: CPU performs transfer

CPU completes its bus transaction.

Step 4: CPU releases bus

The arbiter checks remaining requests.

DMA is the highest-priority remaining requester.

Step 5: Grant DMA



DMA receives the bus and performs its transfer.

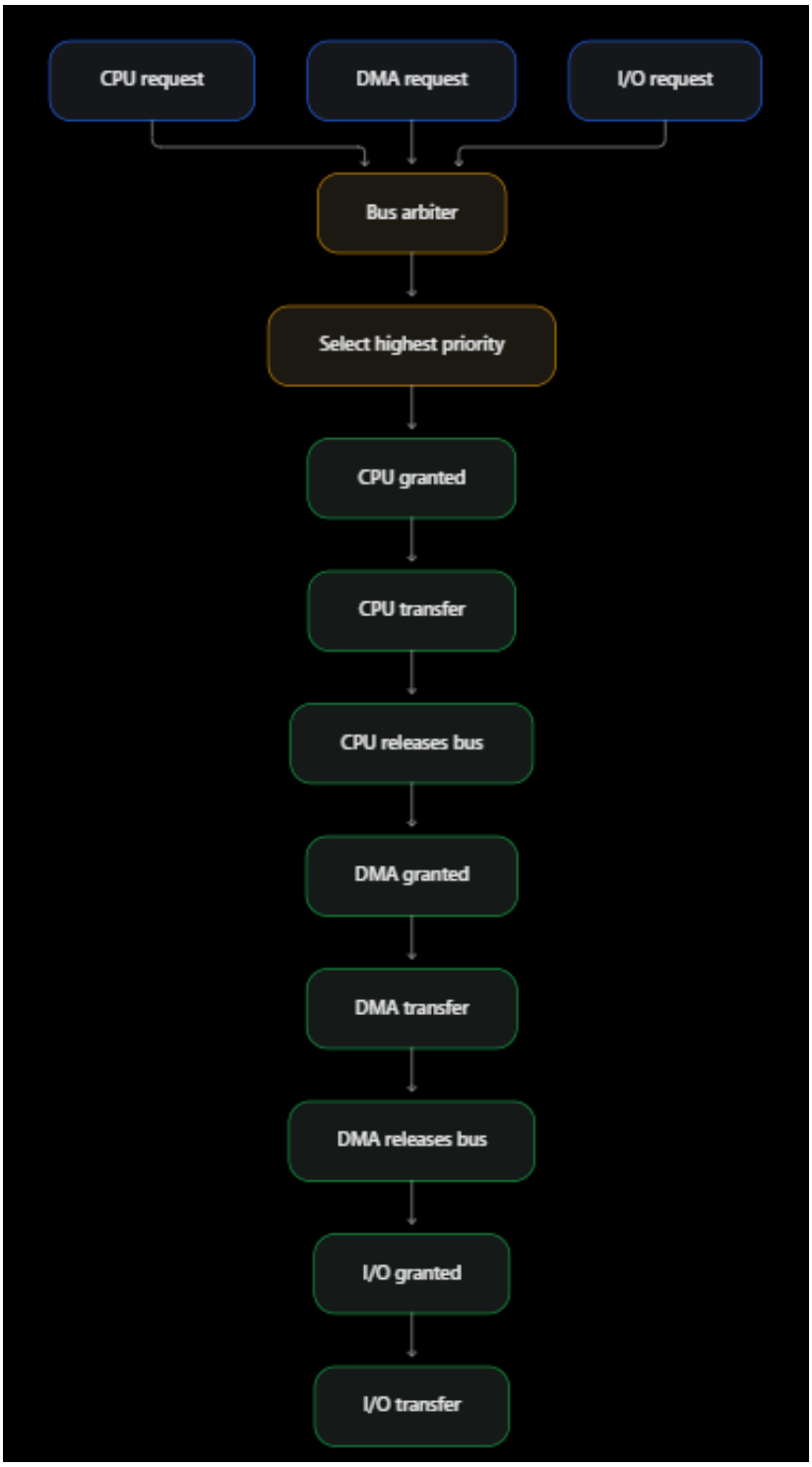
Step 6: Grant I/O

After DMA releases the bus, the I/O controller receives access.

Bus allocation table

Bus cycle	CPU	DMA	I/O	Selected master
1	Request	Request	Request	CPU
2	Released	Request	Request	DMA
3	Released	Released	Request	I/O

Allocation diagram



Advantages of bus arbitration

1. Prevents simultaneous bus use.
2. Avoids transfer conflicts.
3. Supports multiple bus masters.
4. Provides controlled resource allocation.

Disadvantages

1. Arbitration introduces delay.
2. Fixed priority can cause starvation of low-priority masters.
3. Additional arbitration hardware or control logic is required.
4. Bus contention may reduce system performance.

Conclusion

Bus arbitration selects one master from multiple requesting devices. Fixed-priority arbitration uses assigned priority, daisy-chain arbitration uses physical position, and round-robin arbitration rotates access.

14	Data transfer	Processor, memory, I/O, bus
15	Arbitration	Fixed priority, daisy chain, round robin

Exam preparation tip

For an 8–12 mark answer:

1. Write the definition.
2. Draw a neat, labeled diagram when required.
3. Explain the operation in numbered steps.
4. Include a suitable example or trace table.
5. Add advantages and disadvantages if relevant.
6. End with a short conclusion.

For numerical or tracing questions, always show initial values → instruction-by-instruction operation → final values.